HOMEWORK FOUR SOLUTION– CSE 355

DUE: 07 APRIL 2011

Please note that there is more than one way to answer most of these questions. The following only represents a sample solution.

(1) **2.36: A non context free language that works in the pumping lemma**

Solution:

Let $F = \{a^i b^j c^k d^m | i, j, k, m \geq 0 \text{ and if } i = 1 \text{ then } j = k = m\}$. We will show F is not context free, but F works in the pumping lemma.

First we'll show that F is not context free. For a contradiction assume F is context free, then $F \cap \{ab^i c^j d^k | i, j, k \geq 0\} = \{ab^i c^i d^i | i \geq 0\} = G$ is context free since $\{ab^i c^j d^k | i, j, k \geq 0\}$ is the language of the regular expression $ab^*c^*d^*$, it is regular, and as Problem 2.18 in Sipser (whose proof is given in the book) shows, a context free language intersected with a regular language is context free. We will now use the pumping lemma to show that G cannot be context free, which will contradict the fact the we assumed F is context free. Let p be the pumping length of G and take $s = ab^p c^p d^p \in G$ with $|s| > p$. Then there exists u, v, x, y, z such that $s = uvxyz$ and (1) $uv^n xy^n z \in G$ for all $n \geq 0$, (2) $|vy| > 0$ and (3) $|vxy| \leq p$. We will show that for any valid choice of $uvxyz$ that $uv^2xy^2z \notin G$.

Case 1: v or y contains the a. Then uv^2xy^2z will have more than one a and thus in not in G.
Case 2: v and y do not contain the a. In this case from (3) vxy can contain at most two of the other symbols b, c or d. Therefore, uv^2xy^2z will not have the same number of the three symbols.

Now we'll show that F acts like a context free language in the pumping lemma. First we must give a p. Take $p = 2$ although any $p \geq 1$ will work. Now we must show for any string $s \in F$, with $|s| \geq 2$, can be written as $uvxyz$ such that (1) $uv^n xy^n z \in G$ for all $n \geq 0$, (2) $|vy| > 0$ and (3) $|vxy| \leq p$. Again, we will proceed by cases.

Case 1: $s = a^i b^j c^k d^m$ with $i \neq 2$ and $i + j + k + m \geq 2$. In this case, take $u = v = x = \epsilon$, y to be the first symbol in s and z to be the remaining symbols. Then (2) and (3) hold, and $uv^n xy^n z$ will have either zero as ($i = 0$ or $i = 1$ and $n = 0$) or more than one a followed by a string of the form $b^j c^k d^m$, so it will remain in F and we see (1) holds.
Case 2: $s = a^2 b^j c^k d^m \in F$ for some $j, k, m \geq 0$ (so $|s| \geq 2$). Then take $u = v = x = \epsilon$, $y = a^2$ (a won't work here, because of pumping down if any $j, k, m > 0$, but in that case taking the first symbol not an a would work), and $z = b^j c^k d^m$. Then (2) and (3) hold and $xy^i z = a^{2+2(i-1)} b^j c^k d^m \in F$, so (1) also holds.

Thus, in either case we see that the conditions of the pumping lemma hold. Therefore F satisfies the pumping lemma.

(2) **2.31 and 2.32: Pumping Lemma for CFLs**

Solution:

2.31 $B = \{w | w$ is a palindrome over $\{0,1\}$ and w contains an equal number of 0s and 1s$\}$

For a contradiction assume that B is context free. Therefore, B has a pumping length p. Take $s = 0^p 1^{2p} 0^p \in B$ with $|s| > p$. Therefore, there exists $uvxyz$ such that (1) $uv^i xy^i z \in B$ for all $i \geq 0$, (2) $|vy| > 0$ and (3) $|vxy| \leq p$. We will now proceed by cases to show that no matter the values of $uvxyz$ we choose, we will reach a contradiction.

Case 1: vxy consists of only 1s. Then $uv^2xy^2z \notin B$, since it will no longer have the same number of 0s and 1s.
Case 2: vxy contains at least one 0. Then $uv^2xy^2z \notin B$, since it will no longer be a palindrome. This is because from (3), vxy can only contain symbols from the starting 0s or the final 0s, but not both. Thus, after pumping s will have a different number 0s before and after the 1s.

From (2) these are all the cases. In each case we contradict (1). Therefore, B is not context free.

2.32 $C = \{w \in \{0,1,2,3,4\}^* |$ in w the number of 1s equals the number of 2s and the number of 3s equals the number of 4s $\}$

For a contradiction assume that C is context free. Therefore, C has a pumping length p. Take $s = 1^p 3^p 2^p 4^p \in C$ with $|s| > p$. Therefore, there exists $uvxyz$ such that (1) $uv^i xy^i z \in C$ for all $i \geq 0$, (2) $|vy| > 0$ and (3) $|vxy| \leq p$. We will now proceed by cases to show that no matter the values of $uvxyz$ we choose, we will reach a contradiction.

Case 1: vxy contains a 1. Then $uv^2xy^2z \notin C$, since it will no longer have the same number of 1s and 2s. This is because from (3), vxy cannot contain any 2s.
Case 2: vxy contains a 2. Then $uv^2xy^2z \notin C$, since it will no longer have the same number of 1s and 2s. This is because from (3), vxy cannot contain any 1s.
Case 1: vxy contains a 3. Then $uv^2xy^2z \notin C$, since it will no longer have the same number of 3s and 4s. This is because from (3), vxy cannot contain any 4s.
Case 1: vxy contains a 4. Then $uv^2xy^2z \notin C$, since it will no longer have the same number of 3s and 4s. This is because from (3), vxy cannot contain any 3s.

From (2) we see that these are all the cases. In each one we contradict (1). Therefore, C is not context free.

(3) **2.44: Show that if A and B are regular then $A \diamond B = \{xy | x \in A$ and $y \in B$ and $|x| = |y|\}$ is context free**

Solution:

We will construct a PDA, M, that will recognize $A \diamond B$. Since A and B are regular there are DFAs D_A and D_B such that $L(D_A) = A$ and $L(D_B) = B$. Informally we construct the M as follows. From a new start state we first push a \$ onto the stack to mark the stack's bottom and transitions to the start state of D_A. Then for every symbol it sees from A it pushes a 0 onto the stack for counting. It nondeterministically guesses the end of the string from A and transitions to the start symbol of D_B if it is in a final state of D_A. It then pops a 0 for every symbol from the string in B and only moves the accepting state if it is currently in a final state of D_B and is at the bottom of the stack.
Clearly this will recognize $A \diamond B$ since we nondeterministically guess the middle of the string and keep track with our stack to ensure that $|x| = |y|$.

Only the informal description is required, but for completeness we will formally define M. Formally, we define M as follows. We have $D_A = (Q_A, \Sigma, \delta_A, q_A, F_A)$ and $D_B = (Q_B, \Sigma, \delta_B, q_B, F_B)$. Define $M = (Q, \Sigma, \Gamma, \delta, q_0, F)$ where:

- $Q = Q_A \cup Q_B \cup \{q_0, q_f\}$, the union of states from D_A and D_B, plus a new start state and final state.
- Σ is the same for all languages.
- $\Gamma = \{\$, 0\}$, \$ to mark the beginning of the stack and 0 to count how many symbols have been seen in the string from A.
- $$\delta(q,a,b) = \begin{cases} \{q_a, \$\} & \text{if } q = q_0,\ a = b = \epsilon \\ \{\delta_A(q,a), 0\} & \text{if } q \in Q_A,\ a \in \Sigma,\ b = \epsilon \\ \{q_B, \epsilon\} & \text{if } q \in F_A,\ a = b = \epsilon \\ \{\delta_B(q,a), \epsilon\} & \text{if } q \in Q_B,\ a \in \Sigma,\ b = 0 \\ \{q_f, \epsilon\} & \text{if } q \in F_B,\ a = \epsilon,\ b = \$ \\ \emptyset & \text{otherwise} \end{cases}$$
  The transition function works as informally described above.
- $F = \{q_f\}$, the only final state.

To formally prove that $L(M) = A \diamond B$, we will use the idea of configurations (Note a configuration is a 3-tuple (x, q, y) where $x \in \Sigma^*$ is the input string remaining, $q \in Q$ is the current state, and $y \in \Gamma^*$ is the string on the stack, and $c \vdash d$ means configuration d follows configuration c by a transition). If $w \in A \diamond B$ then $w = xy$ with $x \in A$, $y \in B$, and $|x| = |y|$. An accepting computation of w then progresses through the following valid configurations: $(xy, q_0, \epsilon) \vdash (xy, q_A, \$) \vdash^* (y, q_{fA}, 0^{|x|}\$) \vdash (y, q_B, 0^{|x|}\$) \vdash^* (\epsilon, q_{fB}, \$) \vdash (\epsilon, q_f, \epsilon)$, where $q_{fA} \in F_A$ and $q_{fB} \in F_B$.

Conversely, if $w \notin A \diamond B$ then either $xy \notin AB$, the concatenation of A with B, or for all x, y with $x \in A$, $y \in B$ and $w = xy$, $|x| \neq |y|$. In the former case, when xy is consumed M will not be in a final state of D_B and would not be able to transition to the accepting state of M. Therefore, w would not be accepted by M. In the latter case, if $|x| < |y|$ then if M is in an accepting state of D_B after emptying the stack, M will transition to its final state without having consumed all input in which case it will be rejected (transition to $\emptyset$). If M is not in a final state of D_B, M can only transition to the $\emptyset$ again since it has to pop a 0 to transition anywhere else. Alternatively, if $|x| > |y|$, then when the input is consumed M will be in a final state of D_B with 0's remaining on the stack and hence it can only

transition to the $\emptyset$ or remain where it is, in a non-final state of M. Thus, in every case, if $w \notin A \diamond B$, then w is not accepted by M.

Thus, $L(M) = A \diamond B$.

(4) **2.45: Show $A = \{wtw^{\mathcal{R}} | w, t \in \{0,1\}^*$ and $|w| = |t|\}$ is not context free**

Solution:

First, note that since every string $s = wtw^{\mathcal{R}} \in A$ has $|w| = |t| = |w^{\mathcal{R}}|$, $|s|$ is a multiple of 3. For a contradiction assume that A is context free. Therefore, A has a pumping length p. Take $s = 0^{2p}0^p1^p0^{2p} \in A$ with $|s| > p$. Therefore, there exists $uvxyz$ such that (1) $uv^ixy^iz \in B$ for all $i \geq 0$, (2) $|vy| > 0$ and (3) $|vxy| \leq p$. We will now proceed by cases to show that no matter the values of $uvxyz$ we choose, we will reach a contradiction.

Case 1: $|vy|$ is not a multiple of 3. Then $s' = uv^2xy^2z \notin A$ since $|s'|$ will no longer be a multiple of 3.
Case 2: vxy consists of only 0s from the prefix set of 0s and $|vy| = 3r$ for some r. Then, $uv^2xy^2z = 0^{3p+3r}1^p0^{2p} = 0^{2p+r}0^{p+2r}1^{p-r}1^r0^{2p} \notin A$, since $w = 0^{2p+r}$ and $w^{\mathcal{R}} \neq 1^r0^{2p}$, the final third of the string.
Case 3: vxy consists of only 1s and $|vy| = 3r$ for some r. Then, $uv^2xy^2z = 0^{3p}1^{p+3r}0^{2p} = 0^{2p+r}0^{p-r}1^{p+2r}1^r0^{2p} \notin A$, since $w = 0^{2p+r}$ and $w^{\mathcal{R}} \neq 1^r0^{2p}$, the final third of the string.
Case 4: vxy consists of only 0s from the suffix set of 0s and $|vy| = 3r$ for some r. Then, $uv^0xy^0z = 0^{3p}1^p0^{2p-3r} = 0^{2p-r}0^{p+r}1^{p-2r}1^{2r}0^{2p-3r} \notin A$, since $w = 0^{2p-r}$ and $w^{\mathcal{R}} \neq 1^{2r}0^{2p-3r}$, the final third of the string.
Case 5: $vy = 0^m1^n$ with $m, n > 0$ and $m + n = 3r$ for some r. Then, $uv^2xy^2z = 0^{3p+m}1^{p+n}0^{2p} = 0^{2p+r}0^{p+m-r}1^{p+n-r}1^r0^{2p} \notin A$ with $s + t = 2p + r$, since $w = 0^{2p+r}$ and $w^{\mathcal{R}} \neq 1^r0^{2p}$, the final third of the string.
Case 6: $vy = 1^m0^n$ with $m, n > 0$ and $m + n = 3r$ for some r.
Sub-case 6.1: $n < r$. Then, $uv^2xy^2z = 0^{3p}1^{p+m}0^{2p+n} = 0^{2p+r}0^{p-r}1^{p+m+n-r}1^{r-n}0^{2p+n} \notin A$, since $w = 0^{2p+r}$ and $w^{\mathcal{R}} \neq 1^{r-n}0^{2p+n}$, the final third of the string.
Sub-case 6.2: $n > r$. Then, $uv^0xy^0z = 0^{3p}1^{p-m}0^{2p-n} = 0^{2p-r}0^{p+r}1^{p+r-m-n}1^{n-r}0^{2p-n} \notin A$, since $w = 0^{2p-r}$ and $w^{\mathcal{R}} \neq 1^{n-r}0^{2p-n}$, the final third of the string.
Sub-case 6.3: $n = r$. Then, $uv^{p+2}xy^{p+2}z = 0^{3p}1^{p+2r(p+2-1)}0^{2p+r(p+2-1)} = 0^{3p}1^{rp+r-p}1^{2p+rp+r}0^{2p+rp+r} \notin A$, since $w = 0^{3p}1^{rp+r-p}$ and $w^{\mathcal{R}} \neq 0^{2p+rp+r}$, the final third of the string. Note, if we took $i < p + 2$, then taking $r = 1$, $uv^ixy^iz = 0^{3p}1^{p+2(i-1)}0^{2p+(i-1)} = 0^{2p+(i-1)}0^{p-(i-1)}1^{p+2(i-1)}0^{2p+(i-1)} \in A$, since there are not enough 1s to force them into w. $p + 2$ is the first time that is guaranteed.
Thus, we see that in every case (1) is contradicted. Thus, A is not a context free language.

(5) A context-free grammar is in Greibach normal form if every rule has exactly one terminal, followed by zero or more variables, on the right hand side – except when the variable on the left hand side is the start variable, we also allow a rule with the right hand side being the empty string. Show that for every context-free language, there is a context-free grammar in Greibach normal form that generates the language.

Solution:

A context-free grammar $G = (V, \Sigma, R, S)$ is in Greibach normal form if every rule is of one of the following forms:

$$\begin{array}{lcl} S & \rightarrow & \epsilon \\ A & \rightarrow & aX \quad \text{with } A \in V,\ a \in \Sigma \text{ and } X \in V^* \end{array}$$

Let L be a context-free language, then there exists a context-free grammar $G = (V = \{A_0, A_1, \ldots, A_n\}, \Sigma, R, A_0)$ in Chomsky normal form such that $L(G) = L$. Then we already have that the only rule with the empty string on the right hand side is the start variable, and the rules of the form $A \rightarrow a$ for $A \in V$, $a \in \Sigma$ are already in Greibach normal form. We only need to transform rules of the form $A \rightarrow BC$ to Greibach normal form. First we will transform the rules so that if $A_i \rightarrow A_j\alpha \in R$, then $i < j$ (One special issue is handling rules with left-recursion. This is described in the algorithm below). This forces all rules with A_n on the left hand side to be in the proper form. We can then use back substitution to finish the transformation. We do so with the following algorithm:

- For $i = 1$ to n: //(A_0 is already taken care of for this part by the CNF)
  - For $j = 1$ to $i - 1$:
    - If $A_i \rightarrow A_jA \in R$ with $i > j$ and $A \in V$
      - For each rule $A_j \rightarrow \alpha \in R$ with $\alpha \in (V \cup \Sigma)^*$:
      (a) Add the rule $A_i \rightarrow \alpha A$ to R
      - Remove $A_i \rightarrow A_jA$ from R
  - //Now we will take care of left hand recursion rules
  - If $A_i \rightarrow A_i\alpha \in R$ for some $\alpha \in (V)^*$:
    - Add a new variable B_i to V
    - For each $A_i \rightarrow A_i\alpha \in R$ for some $\alpha \in (V)^*$:
      - Add the rule $B_i \rightarrow \alpha B_i$ to R
      - Add the rule $B_i \rightarrow \alpha$ to R
      - Remove the rule $A_i \rightarrow A_i\alpha$ from R
    - If $A_i \rightarrow \alpha \in R$ with $\alpha \in (V \cup \Sigma)^*$:
      - Add the rule $A_i \rightarrow \alpha B_i$ to R
- //Now A_n is in proper form and each A_i with $i < n$ has either terminals or variables with a larger subscript. Therefore we will use back substitution to to turn the remaining A_i rules to the proper form.
- For $i = n - 1$ down to 0:
  - For each rule $A_i \rightarrow A_j\alpha \in R$ with $j > i$ and $\alpha \in (V \cup \Sigma)^*$:
    - For each rule $A_j \rightarrow a\beta \in R$ with $a \in \Sigma$ and $\beta \in (V \cup \Sigma)^*$:
      - Add the rule $A_i \rightarrow a\beta\alpha$ to R
    - Remove the rule $A_i \rightarrow A_j\alpha$ from R
- //Now we only have to take care of the rules with B_i on the left hand side
- For $i = 1$ to n:
  - For each rule $B_i \rightarrow A_j\alpha B_i \in R$ with $\alpha \in (V \cup \Sigma)^*$:
    - For each rule $A_j \rightarrow a\beta \in R$ with $a \in \Sigma$ and $\beta \in (V \cup \Sigma)^*$:

- Add the rule $B_i \to a\beta\alpha B_i$ to R
* Remove the rule $B_i \to A_j\alpha B_i$ from R

After applying the algorithm we end up with an equivalent grammar in Greibach normal form since we only replace variables with all possible strings in $(V \cup \Sigma)^*$ that they could derive in one step, or switch around a recursion. Doing either of these actions do not change the language the grammar recognizes.